CS3001 Computer Networks – Assignment-II

Reliable Transfer for a Disaster-Response Network: GBN vs Selective Repeat

Muhammad Hassan Siddiqui | Roll No: 24K-1051 | Random seed: 1051 | Fall 2026

1. Approach

A satellite link (R = 2 Mbps, RTT = 500 ms, L = 8,000 bits) connects the Emergency Operations Center to field teams. Sending one packet takes only L/R = 4 ms, but its ACK takes 500 ms to return, so the protocol must be pipelined, and because the link loses, corrupts and delays packets and ACKs, every packet must be checked, numbered, acknowledged and retransmitted when needed. I implemented Go-Back-N (GBN) and Selective Repeat (SR) in Python as a discrete-event simulation with my own unreliable channel and no transport library. Runs are reproducible from the seed. The report uses slide terminology (base, nextseqnum, expectedseqnum, rcv_base, cumulative ACK).

2. Task 1: Scenario and Protocol Decision

1. Why is reliable data transfer important in this network?

The network carries casualty lists, supply requests and rescue assignments, where a single lost or corrupted packet could send a team to the wrong location. The satellite link loses, corrupts and delays both packets and ACKs, so the underlying channel cannot be trusted. The transport protocol must therefore detect errors (checksums), preserve ordering (sequence numbers) and recover from loss (ACKs, timeouts and retransmissions) so that every packet arrives correctly and in order.

2. Why can Stop-and-Wait be inefficient on this link?

The sender transmits one packet and must wait for its ACK before sending the next. Sending takes only L/R = 4 ms, but the satellite RTT is 500 ms, so the sender is idle almost all the time. Utilization is U = 0.004 / (0.5 + 0.004) ≈ 0.8%, which wastes nearly all of the limited 2 Mbps bandwidth, and every lost packet or ACK adds a further timeout delay.

3. Why is pipelining useful here?

The 500 ms RTT is far longer than the 4 ms needed to transmit one packet, so a Stop-and-Wait sender would sit idle almost all the time. With pipelining the sender transmits a window of N packets before the first ACK returns, keeping the link busy during the wait. For N = 4 utilization rises from about 0.8% to about 3.2%, so critical data is delivered faster over the limited bandwidth.

4. Is GBN or SR more suitable? Two technical reasons.

SR is more suitable. First, bandwidth is limited to 2 Mbps, and under GBN a single loss forces the sender to retransmit every outstanding packet, while SR retransmits only the missing one, saving scarce capacity. Second, with a 500 ms RTT and a channel that delays packets and ACKs, out-of-order arrivals are common: GBN discards them and resends the whole window, whereas SR buffers them, retransmits only what is actually missing, and still delivers data in order.

3. Task 2: GBN vs SR Packet Trace

Assumptions. Sequence numbers 0–9 (no wrap), N = 4, initial window [0–3]. Packet 2 is lost on its first transmission only. The window slides as soon as an ACK arrives, so pkts 4 and 5 are sent before the timeout and also arrive out of order. GBN has one timer (for base); SR has one timer per packet. Running the simulator with `--force-drop 2` reproduces both traces.

(a) Go-Back-N (cumulative ACKs, receiver discards out-of-order packets)

Ev	Sender	Packet	Receiver	ACK	Window
1	Send pkt 0 (start timer)	0	–	–	[0–3], nextseqnum=1
2	Send pkt 1	1	–	–	nextseqnum=2
3	Send pkt 2	2 **LOST**	–	–	nextseqnum=3
4	Send pkt 3	3	–	–	nextseqnum=4 (full)
5	–	0	Deliver 0, expectedseqnum=1	ACK0	–
6	Recv ACK0, send pkt 4	4	–	–	base=1 → [1–4]
7	–	1	Deliver 1, expectedseqnum=2	ACK1	–
8	Recv ACK1, send pkt 5	5	–	–	base=2 → [2–5]
9	–	3, 4, 5	Out of order (expected 2): **discard all three**	ACK1 ×3 (dup)	–
10	Duplicate ACK1s ignored	–	–	–	base stays 2
11	**TIMEOUT** on base (pkt 2): go back, resend 2, 3, 4, 5	2, 3, 4, 5	–	–	[2–5]
12	–	2, 3, 4, 5	Deliver 2, 3, 4, 5 in order; expectedseqnum=6	ACK2–ACK5	–
13	Recv ACK2–ACK5; each ACK slides window and sends next pkt	6, 7, 8, 9	–	–	base 3→6, [6–9]
14	–	6, 7, 8, 9	Deliver 6–9; expectedseqnum=10	ACK6–ACK9	–
15	Recv ACK6–ACK9, stop timer	–	All 10 delivered	–	base=10

(b) Selective Repeat (individual ACKs, receiver buffers out-of-order packets; S = sender, R = receiver window)

Ev	Sender	Packet	Receiver	ACK	Window
1	Send pkts 0, 1, 2, 3 (timer per packet)	0, 1, **2 LOST**, 3	–	–	S [0–3] / R [0–3]
2	–	0	Deliver 0, rcv_base=1	ACK0	R [1–4]
3	Recv ACK0, mark 0, send pkt 4	4	–	–	S [1–4]
4	–	1	Deliver 1, rcv_base=2	ACK1	R [2–5]
5	Recv ACK1, mark 1, send pkt 5	5	–	–	S [2–5]
6	–	3	In window: **buffer** 3 (rcv_base stays 2)	ACK3	R [2–5]
7	–	4, 5	**Buffer** 4 and 5	ACK4, ACK5	R [2–5]
8	Recv ACK3, 4, 5: mark them ACKed; send_base stuck at 2	–	–	–	S [2–5]
9	**TIMEOUT** on pkt 2 only: resend pkt 2	2	–	–	S [2–5]
10	–	2	Deliver 2, then buffered 3, 4, 5; rcv_base=6	ACK2	R [6–9]
11	Recv ACK2: send_base 2→6; send 6, 7, 8, 9	6, 7, 8, 9	–	–	S [6–9]
12	–	6, 7, 8, 9	Deliver each; rcv_base=10	ACK6–ACK9	–
13	Recv ACK6–ACK9	–	All 10 delivered	–	send_base=10

Retransmissions: GBN = 4 packets (2, 3, 4, 5); SR = 1 packet (2). The GBN receiver discards 3, 4 and 5 and only sends cumulative ACKs, so the sender keeps seeing ACK1 and cannot know they arrived; after the timeout it must resend everything from base onward. The SR receiver buffers 3, 4 and 5 and ACKs each one individually, so only pkt 2's timer expires and only pkt 2 is resent; the buffered packets are delivered in order the moment it arrives. (If pkt 5 were not sent before the timeout, GBN would retransmit 3 packets and SR still 1.)

4. Implementation

The program (`rdt.py`) is a discrete-event simulator: a clock and a priority queue of events (send, data arrival, ACK arrival, timeout) make runs fast and exactly reproducible. Each packet carries a type, sequence number (mod 2^k), payload and a 16-bit Internet checksum. All packets and ACKs pass through one channel function that can **lose** a packet (probability `loss`), **corrupt** it by flipping a random bit, which the receiver or sender detects with the checksum and discards, and **delay** it by extra time, which causes reordering. Every send, receive, ACK, loss, corruption, timeout and retransmission is logged. Parameters: protocol, packets, window N, sequence bits k, loss, corruption, delay and timeout. ACK and window decisions use modular arithmetic, e.g. offset = (seq − base mod 2^k) mod 2^k, and the program rejects unsafe windows (GBN needs N ≤ 2^k − 1, SR needs N ≤ 2^(k−1)) unless `--force` is used.

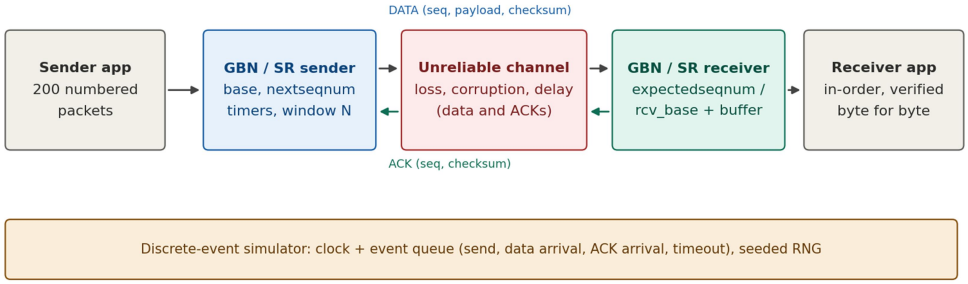


Figure 1: Simulator architecture. Data and ACKs both cross the unreliable channel.

Feature	Go-Back-N	Selective Repeat
Sender state	base, nextseqnum	base, nextseqnum, per-packet ACKed flags
Timers	One timer, for the oldest unACKed packet	One timer per packet
ACKs	Cumulative (ACKn = all up to n)	Individual (one per packet)
Receiver	expectedseqnum; discards out-of-order, re-sends last ACK	rcv_base window; buffers out-of-order, delivers in order
On timeout	Resend base … nextseqnum−1	Resend only the timed-out packet

Timeout design. The timeout must exceed the worst-case round trip (RTT + 2 × maximum extra delay). Testing showed why: with a shorter timeout, a delayed ACK caused a premature retransmission, and the receiver's duplicate ACK arrived after the sender had reused the same 3-bit sequence number, so a stale ACK was accepted as new and data was lost. With the timeout set safely, 960 randomized runs (both protocols, up to 20% loss, 10% corruption, 30% delay) all delivered every packet correctly and in order.

5. Task 4: Experiments and Comparison

Setup: 200 packets, N = 4, k = 3, seed 1234, timeout 0.9 s, same for every run. A = reliable channel; B = loss 0.10; C = loss 0.05, corruption 0.05, delay 0.10. Time is simulated time to deliver all 200 packets; throughput = delivered bits / time. All runs delivered 200/200 packets in order and verified byte for byte.

Scenario	Protocol	Packets transmitted	Retransmissions	Delivered	Time (s)	Throughput (kbps)
A: Reliable	GBN	200	0	200	25.21	63.5
A: Reliable	SR	200	0	200	25.21	63.5
B: Loss 0.10	GBN	327	127	200	60.24	26.6
B: Loss 0.10	SR	253	53	200	52.47	30.5
C: Mixed	GBN	344	144	200	69.94	22.9
C: Mixed	SR	242	42	200	46.52	34.4

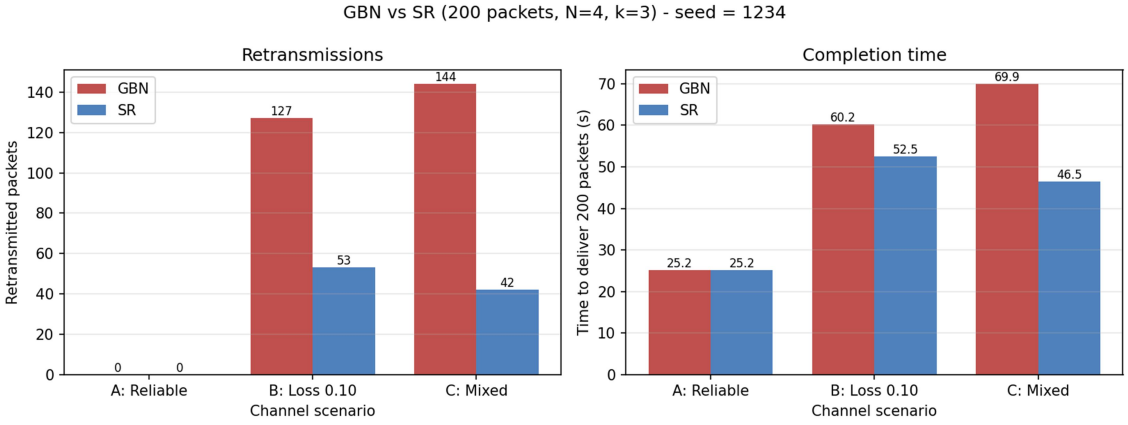


Figure 2: GBN vs SR – retransmissions (left) and completion time (right) for scenarios A, B and C.

Comparison. With a reliable channel (A) the two protocols behave identically because neither needs to retransmit. Under loss (B) and mixed errors (C) SR retransmits far fewer packets (53 vs 127 and 42 vs 144) because it resends only the packet whose timer expired, while GBN resends the whole outstanding window after one loss and also discards every out-of-order packet that delay or loss produces. SR therefore finishes sooner and reaches higher throughput in the error scenarios (46.5 s vs 69.9 s in C), at the price of a more complex sender and receiver with per-packet timers and a buffer. The time gap is smaller than the retransmission gap because completion time is dominated by waiting for timeouts, not by the 4 ms needed to send each packet; averaged over 30 seeds, scenario B times were almost equal while SR's retransmission advantage held in every run.

6. Task 5: Transport Layer Analysis

1. Pipelining and link utilization

Stop-and-Wait leaves the link idle while waiting for each ACK; pipelining fills that waiting time with more packets. With L/R = 8,000 / 2,000,000 = 0.004 s and RTT = 0.5 s:

- **Stop-and-Wait:** U = (L/R) / (RTT + L/R) = 0.004 / 0.504 ≈ **0.0079 (0.79%)**
- **Pipelined, N = 4:** U = N × (L/R) / (RTT + L/R) = 4 × 0.004 / 0.504 = 0.016 / 0.504 ≈ **0.0317 (3.17%)**

Pipelining is about four times better, and my simulation measured the same 3.17% on the reliable channel (scenario A). Utilization is still low because filling the pipe would need N ≈ 504 / 4 = 126 packets, far above what 3-bit numbers allow.

2. Sequence numbers (SR, 3-bit)

- **(a)** 2^3 = **8** sequence numbers (0–7).
- **(b)** Maximum safe window = 2^(k−1) = **4**, since the sender and receiver windows together must not exceed 2^k (N + N ≤ 8).
- **(c)** With a larger window the receiver cannot tell a retransmitted old packet from a new one that reuses the same number. Figure 3 shows N = 5: after pkts 0–4 are delivered the receiver window is {5, 6, 7, 0, 1}; the ACKs are lost, the sender retransmits old pkt 0, and the receiver accepts it as new data. With N = 4 the window is {4, 5, 6, 7}, so seq 0 is recognised as old, re-ACKed and discarded. Running my SR with N = 5, k = 3 (`--force`) ends with verification failed, confirming the ambiguity.

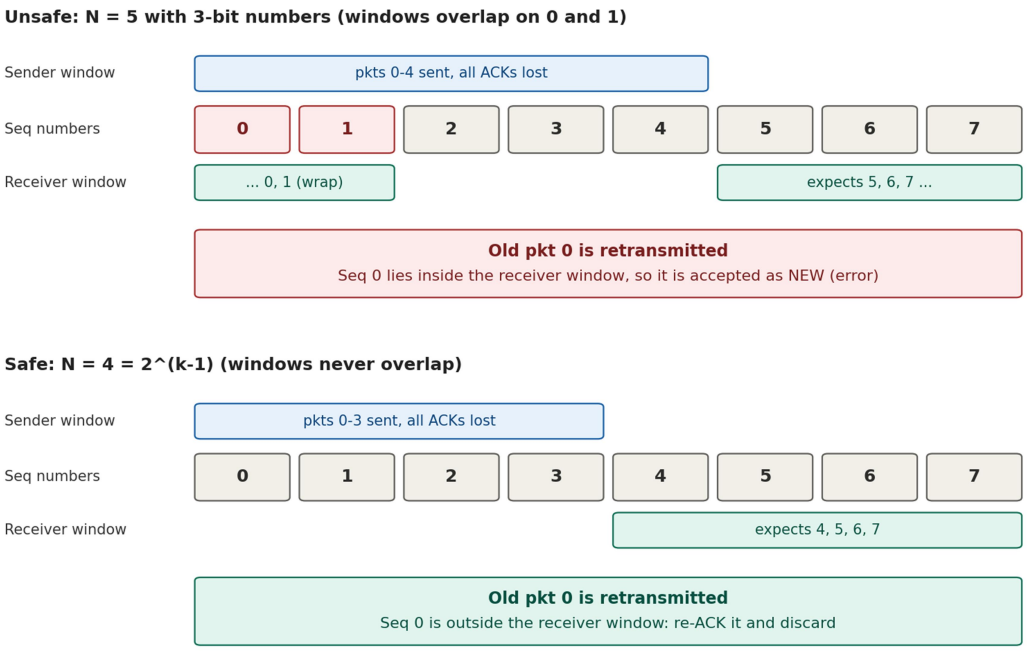


Figure 3: SR with 3-bit sequence numbers. Window N = 5 overlaps (ambiguous); N = 4 does not (safe).

7. Conclusion

Both protocols deliver every packet correctly and in order over a lossy, corrupting and delaying satellite link, and pipelining lifts utilization from 0.79% to 3.17% with N = 4. SR is the better fit for this network: it retransmits 2–3 times fewer packets than GBN under errors and finishes sooner, because it buffers out-of-order packets and resends only what is missing. The trade-offs are SR's extra complexity and its stricter limit on window size ($N \leq 2^{(k-1)}$), plus the need for a timeout longer than the worst-case round trip so that stale duplicates cannot be mistaken for new packets.